РОССИЙСКАЯ ФЕДЕРАЦИЯ
ХАНТЫ-МАНСИЙСКИЙ АВТОНОМНЫЙ ОКРУГ-ЮГРА

АВТОНОМНАЯ НЕКОММЕРЧЕСКАЯ ПРОФЕССИОНАЛЬНАЯ ОБРАЗОВАТЕЛЬНАЯ ОРГАНИЗАЦИЯ
«СУРГУТСКИЙ ИНСТИТУТ ЭКОНОМИКИ, УПРАВЛЕНИЯ И ПРАВА»

09.02.07 Информационные системы и программирование

КУРСОВАЯ РАБОТА

по дисциплине «МДК.09.01 Проектирование и разработка веб-приложений»

на тему: «Проектирование и разработка веб-приложения пиццерии: меню, корзина и статус заказа»

Выполнил: Матвеев Е.К.
студент группы ИС24/11
Руководитель: Преподаватель профессиональных дисциплин
Мацюк Н.А._____
Оценка _____
Подпись_____
Дата _____

Сургут, 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3
ГЛАВА 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ РАЗРАБОТКИ ВЕБ-ПРИЛОЖЕНИЯ ПИЦЦЕРИИ 6
1.1 Анализ предметной области и существующих решений 6
1.2 Технологии и средства разработки веб-приложений 8
1.3 Требования к веб-приложению пиццерии 12
ГЛАВА 2. ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ ПИЦЦЕРИИ 16
2.1 Проектирование архитектуры и базы данных 16
2.2 Разработка серверной и клиентской частей 20
2.3 Тестирование веб-приложения 31
ЗАКЛЮЧЕНИЕ 35
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 37
ПРИЛОЖЕНИЯ 40

ВВЕДЕНИЕ

За последние годы заказ еды через интернет стал для большинства горожан привычным делом. Покупателю удобнее открыть сайт, спокойно посмотреть меню с фотографиями и ценами, собрать заказ и видеть, на каком этапе он находится, чем дозваниваться до оператора и диктовать адрес по телефону. Для заведения общественного питания онлайн-заказ тоже выгоден: снижается количество ошибок при приёме заказа, операторы освобождаются от рутинной работы, а сами заказы сразу попадают в систему в структурированном виде.

Небольшие пиццерии чаще всего начинают продавать через крупные агрегаторы доставки. Это быстрый способ выйти в интернет, но у него есть заметные недостатки: агрегатор берёт комиссию с каждого заказа, сам определяет внешний вид витрины и правила работы, а данные о покупателях остаются у площадки. Поэтому многие заведения со временем приходят к собственному веб-приложению, через которое заказы принимаются напрямую.

Актуальность темы курсовой работы определяется тем, что собственное веб-приложение для пиццерии позволяет снизить зависимость от посредников и при этом дать покупателю привычный набор функций: каталог блюд по категориям, корзину и информацию о статусе заказа. Отдельно стоит отметить требование к отслеживанию заказа: покупатели ожидают, что статус будет меняться на экране сам, без постоянного обновления страницы.

Новизна работы заключается в том, что приложение разделено на два независимых сервера. Серверная часть на фреймворке Django и библиотеке Django REST Framework отвечает за базу данных, правила работы с корзиной и заказами и вход пользователей и общается с внешним миром только через программный интерфейс REST API в формате JSON. Клиентская часть – это

отдельный сайт на HTML, CSS и JavaScript, который получает все данные запросами к API: корзина изменяется без перезагрузки страницы, а статус заказа обновляется на экране покупателя автоматически после того, как его изменил администратор. Вход и регистрация выполняются по токену, который выдаёт сервер.

Гипотеза работы: если предоставить покупателю возможность самостоятельно формировать заказ в веб-приложении и наблюдать изменение его статуса в реальном времени, то процесс заказа станет быстрее и прозрачнее для клиента, а нагрузка на персонал пиццерии по приёму заказов и ответам на вопросы «где мой заказ» снизится.

Цель курсовой работы – спроектировать и разработать веб-приложение пиццерии, которое позволяет просматривать меню, формировать корзину, оформлять заказ и отслеживать его статус.

Для достижения поставленной цели необходимо решить следующие задачи:

– проанализировать предметную область и существующие способы организации онлайн-заказа в сфере общественного питания;

– рассмотреть технологии разработки веб-приложений и обосновать выбор средств разработки;

– сформулировать функциональные и нефункциональные требования к приложению;

– спроектировать архитектуру приложения и структуру базы данных;

– разработать серверную и клиентскую части веб-приложения;

– провести тестирование приложения и оценить полученные результаты.

Объект исследования – процесс приёма и обработки заказов в пиццерии.

Предмет исследования – веб-приложение для онлайн-заказа пиццы, включающее меню, корзину и отслеживание статуса заказа.

В работе использовались следующие методы: анализ учебной литературы, нормативных документов и технической документации, сравнительный анализ аналогов и программных средств, моделирование (диаграмма вариантов использования, диаграмма состояний, ER-диаграмма), объектно-ориентированное программирование, а также модульное и ручное тестирование.

Практическая значимость работы состоит в том, что разработанное приложение готово к использованию небольшой пиццерией: администратор ведёт меню и меняет статусы заказов через панель администрирования, а покупатели оформляют заказы самостоятельно. Исходный код размещён в открытом репозитории и может быть доработан под требования конкретного заведения.

Курсовая работа состоит из введения, двух глав, заключения, списка использованных источников и приложений. Работа оформлена в соответствии с ГОСТ 7.32-2017 [5], список источников составлен по ГОСТ Р 7.0.100-2018 [6]. В первой главе рассматриваются теоретические основы: предметная область, технологии разработки и требования к приложению. Во второй главе описаны проектирование, разработка и тестирование веб-приложения.

ГЛАВА 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ РАЗРАБОТКИ ВЕБ-ПРИЛОЖЕНИЯ ПИЦЦЕРИИ

1.1 Анализ предметной области и существующих решений

Пиццерия относится к предприятиям общественного питания, которые работают как с гостями в зале, так и на вынос и с доставкой. Независимо от канала продаж, работа с заказом проходит одни и те же этапы: покупатель знакомится с меню, выбирает позиции и их количество, сообщает контактные данные и адрес, после чего заказ передаётся на кухню, готовится, передаётся курьеру и доставляется.

При традиционном приёме заказов по телефону большая часть этих этапов зависит от оператора. Оператор зачитывает или уточняет состав блюд, записывает позиции и адрес, называет итоговую сумму. На практике это приводит к типичным проблемам:

– ошибки при записи позиций, адреса или номера телефона;

– очередь звонков в часы пик, когда покупатель вынужден ждать ответа;

– отсутствие у покупателя информации о том, что происходит с заказом, и повторные звонки с вопросом о готовности;

– сложность учёта: заказы оказываются разбросаны по записям операторов.

Веб-приложение решает эти проблемы за счёт того, что покупатель сам формирует заказ на основе актуального меню, а все данные сразу сохраняются в базе данных. Из анализа процесса можно выделить основные понятия предметной области, которые в дальнейшем станут сущностями базы данных: категория меню, товар (пицца, закуска, соус, напиток), корзина и её позиции, заказ, позиции заказа и статус заказа.

Статус заказа является ключевым понятием для покупателя. В работе принята следующая последовательность статусов: «Новый» – заказ оформлен и ожидает обработки; «Готовится» – заказ передан на кухню; «В доставке» – заказ передан курьеру; «Завершён» – заказ доставлен. Отдельно предусмотрен статус «Отменён». Отменить заказ можно, пока его не передали курьеру, то есть в статусах «Новый» и «Готовится».

Существующие способы организации онлайн-заказа для заведений общественного питания можно разделить на три группы. Первая – агрегаторы доставки, например «Яндекс Еда», на которых заведение размещает своё меню. Вторая – собственные сайты и мобильные приложения крупных сетей, например «Додо Пицца», разработанные под конкретную компанию. Третья – сайты, собранные на конструкторах, например Tilda, где интернет-магазин настраивается из готовых блоков. Сравнение этих решений с разрабатываемым приложением приведено в таблице 1.

Таблица 1 – Сравнение способов организации онлайн-заказа

Критерий	Агрегатор доставки	Приложение крупной сети	Сайт на конструкторе	Разрабатываемое приложение
Меню с категориями	есть	есть	есть	есть
Корзина	есть	есть	есть	есть
Отслеживание статуса заказа	есть	есть	нет	есть
Комиссия с каждого заказа	есть	нет	нет	нет
Контроль над данными покупателей	у площадки	у заведения	у заведения	у заведения
Затраты на создание	низкие	очень высокие	низкие	средние
Возможность доработки под заведение	нет	да	ограничена	да

Из таблицы 1 видно, что агрегатор удобен для быстрого старта, но связан с комиссией и не даёт заведению полного контроля над данными. Решения крупных сетей обладают всеми нужными функциями, однако их разработка требует значительных затрат и команды разработчиков.

Конструкторы позволяют дёшево запустить витрину и корзину, но, как правило, не дают покупателю отслеживать статус заказа. Собственное веб-приложение на открытых технологиях занимает промежуточное положение: при умеренных затратах оно обеспечивает весь необходимый набор функций и может дорабатываться под нужды заведения.

При разработке приложения необходимо учитывать требования законодательства. Закон «О защите прав потребителей» обязывает продавца доводить до покупателя необходимую и достоверную информацию о товаре, в том числе о его цене и составе [3]. Поэтому в карточке каждого товара в меню указываются цена, состав и размер или объём. Адрес доставки, номер телефона и имя пользователя относятся к персональным данным, обработка которых регулируется Федеральным законом «О персональных данных» [1]. Из этого следуют требования собирать только те данные, которые нужны для выполнения заказа, и не допускать, чтобы один покупатель видел заказы другого. Общие правила защиты информации в информационных системах установлены Федеральным законом «Об информации, информационных технологиях и о защите информации» [2].

Таким образом, анализ предметной области показал, что для небольшой пиццерии целесообразно собственное веб-приложение, которое реализует каталог блюд по категориям, корзину, оформление заказа и отслеживание его статуса, а также учитывает требования к информации о товаре и к защите персональных данных покупателей.

1.2 Технологии и средства разработки веб-приложений

Веб-приложение построено по клиент-серверной модели. Клиентом выступает браузер пользователя, который отправляет запросы по протоколу HTTP, а сервер обрабатывает их и возвращает ответ [9]. В разрабатываемом приложении серверов два. Сервер бэкенда возвращает только данные в

формате JSON (JavaScript Object Notation) [22] и файлы фотографий, а сервер фронтенда отдаёт браузеру статические страницы, стили и сценарии. Для получения данных используется метод GET, для изменения данных, например добавления товара в корзину или оформления заказа, – метод POST.

Взаимодействие частей построено по архитектурному стилю REST (Representational State Transfer), описанному Р. Филдингом [21]. Каждый ресурс – категория, товар, корзина, заказ – доступен по своему адресу, сервер не хранит состояние клиента между запросами, а результат сообщает кодом ответа: 200 – успешно, 201 – объект создан, 400 – ошибка в данных, 401 – пользователь не вошёл в систему, 403 – доступ запрещён, 404 – объект не найден. Такой интерфейс называют REST API (Application Programming Interface).

Серверная часть современных веб-приложений обычно строится на основе фреймворка – набора готовых компонентов, который берёт на себя типовые задачи: разбор адресов запросов, работу с базой данных, авторизацию пользователей и защиту от распространённых атак. Для выбора фреймворка были рассмотрены четыре распространённых решения: Django и Flask на языке Python, Laravel на PHP и Express на JavaScript (Node.js). Результаты сравнения приведены в таблице 2.

Таблица 2 – Сравнение фреймворков для разработки серверной части

Критерий	Django	Flask	Laravel	Express
Язык программирования	Python	Python	PHP	JavaScript
Встроенная панель администратора	есть	нет, сторонние расширения	нет, сторонние пакеты	нет
Встроенная ORM	есть	нет, используется SQLAlchemy	есть (Eloquent)	нет, сторонние библиотеки
Встроенная система пользователей	есть	нет, расширения	есть	нет, сторонние библиотеки
Создание REST API с входом по токену	Django REST Framework	Flask-RESTful, расширения	встроено (Sanctum)	вручную, сторонние библиотеки
Миграции базы данных	встроены	расширения	встроены	сторонние библиотеки

Для веб-приложения пиццерии был выбран фреймворк Django. Главным доводом стала встроенная панель администратора: она автоматически строится по моделям данных и позволяет без отдельной разработки вести меню и изменять статусы заказов. Кроме того, Django включает систему пользователей с хешированием паролей и проверкой их сложности, объектно-реляционное отображение (ORM) и систему миграций базы данных [7, 15].

Django следует архитектурному шаблону MTV (Model – Template – View), который является вариантом известного шаблона MVC [9, 12]. В разрабатываемом приложении шаблоны страниц на сервере не используются: представление получает запрос, обращается к моделям, передаёт результат сериализатору и возвращает ответ в формате JSON, а за внешний вид полностью отвечает клиентская часть. Такое разделение упрощает разработку: изменения в оформлении не затрагивают логику, и наоборот.

Язык программирования Python выбран благодаря простому и читаемому синтаксису, большому количеству библиотек и хорошей документации [8, 18]. Для разработки программного интерфейса (API) использована библиотека Django REST Framework. Она добавляет к Django сериализаторы, которые преобразуют объекты моделей в JSON и обратно,

наборы представлений (ViewSet), по которым автоматически строятся адреса API, классы прав доступа и аутентификацию по токену TokenAuthentication [14, 23]. Чтобы браузер разрешил странице фронтенда обращаться к бэкенду на другом адресе, подключён пакет django-cors-headers, который добавляет к ответам заголовки CORS (Cross-Origin Resource Sharing) [20].

В качестве системы управления базами данных выбрана SQLite. Это встраиваемая СУБД: база данных хранится в одном файле и не требует отдельного сервера, что удобно на этапе разработки и для небольшого заведения [19]. Благодаря ORM Django при росте нагрузки базу данных можно перевести, например, на PostgreSQL без переписывания кода приложения [7].

Клиентская часть приложения является отдельным сайтом и использует стандартные веб-технологии:

– язык разметки HTML – для каркаса страниц: шапки, подвала, форм и пустых блоков, которые заполняются данными из API;

– каскадные таблицы стилей CSS – для оформления, сеточной вёрстки карточек, анимаций и адаптации под экраны телефонов;

– язык JavaScript – для запросов к API, вывода меню, корзины и заказов, хранения токена входа в localStorage браузера [24] и взаимодействия с сервером без перезагрузки страницы.

Для асинхронных запросов к серверу применяется Fetch API – встроенный в браузер интерфейс, который позволяет отправить HTTP-запрос из JavaScript и обработать ответ, не перезагружая страницу [16]. Такой подход часто называют AJAX. Для плавного появления карточек при прокрутке используется Intersection Observer API, который сообщает, когда элемент попал в видимую область экрана [17]. Отказ от сторонних библиотек на стороне клиента уменьшает объём загружаемого кода и упрощает сопровождение [10, 11].

Для учебного запуска фронтенд раздаётся встроенным в Python простым веб-сервером командой python -m http.server на порту 5500, а бэкенд

работает на порту 8000. Так как клиентская часть состоит только из статических файлов, её можно разместить на любом хостинге или собрать в конструкторе сайтов, например Tilda, сохранив обращения к тем же адресам API. Разработка велась в редакторе кода Visual Studio Code, для хранения исходного кода использовалась система контроля версий Git и сервис GitHub, для отладки клиентской части – инструменты разработчика в браузере.

Итак, для серверной части выбраны Python, Django и Django REST Framework с базой данных SQLite, а для клиентской части – HTML, CSS и JavaScript без сторонних библиотек. Бэкенд и фронтенд работают как два отдельных сервера и общаются только через REST API, что позволяет развивать их независимо и при этом использовать готовые надёжные механизмы безопасности.

1.3 Требования к веб-приложению пиццерии

Требования к приложению сформулированы с учётом рекомендаций ГОСТ 34.602-2020 к составу технического задания на создание автоматизированной системы [4]. По результатам анализа предметной области выделены три роли пользователей:

– гость – посетитель сайта, который ещё не вошёл в систему;

– покупатель – зарегистрированный пользователь, который оформляет заказы;

– администратор – сотрудник пиццерии, который ведёт меню и обрабатывает заказы.

Функциональные требования для каждой роли приведены в таблице 3.

Таблица 3 – Функциональные требования к веб-приложению

Роль	Функция	Описание
Гость	Просмотр меню	Просмотр товаров с фото, составом и ценой, разделённых на категории
Гость	Фильтр по категориям	Показ только выбранной категории: пицца, закуски, соусы или напитки
Гость	Регистрация и вход	Создание учётной записи и вход по имени пользователя и паролю, получение токена
Покупатель	Работа с корзиной	Добавление товаров, изменение количества и удаление позиций без перезагрузки страницы
Покупатель	Оформление заказа	Указание адреса доставки и телефона, проверка корректности номера
Покупатель	Просмотр заказов	Список своих заказов с датой, суммой и текущим статусом
Покупатель	Отслеживание статуса	Автоматическое обновление статуса на странице заказа без перезагрузки
Покупатель	Отмена заказа	Отмена заказа в статусах «Новый» и «Готовится»
Покупатель	Выход из аккаунта	Удаление токена на сервере и в браузере
Администратор	Ведение меню	Добавление и изменение категорий и товаров, загрузка фотографий
Администратор	Обработка заказов	Просмотр заказов и изменение их статуса

Взаимосвязь ролей и функций показана на диаграмме вариантов использования в соответствии с рисунком 1. Гость может просматривать меню и регистрироваться, покупатель дополнительно работает с корзиной и заказами, а администратор управляет меню и статусами заказов.

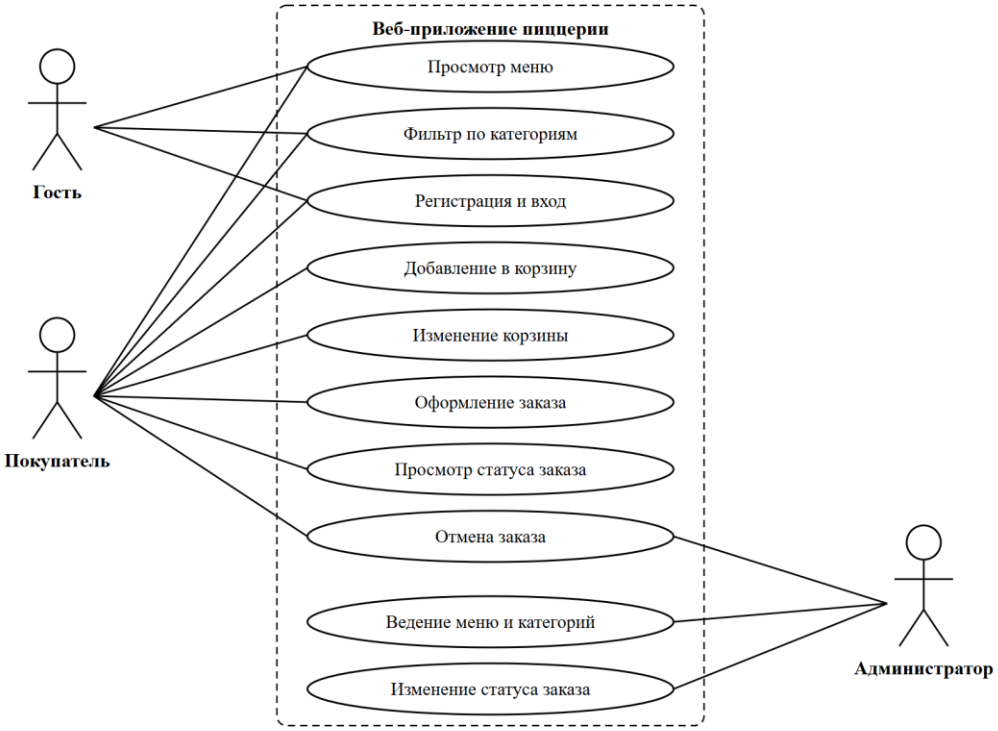


Рисунок 1 – Диаграмма вариантов использования веб-приложения

Помимо функциональных, к приложению предъявляются следующие нефункциональные требования:

– адаптивность: интерфейс должен корректно отображаться на экранах компьютеров и мобильных телефонов;

– отзывчивость: действия с корзиной выполняются без перезагрузки страницы, а результат сразу отображается на экране;

– актуальность статуса: изменение статуса заказа отображается у покупателя не позднее чем через несколько секунд;

– безопасность: пароли хранятся только в виде хешей, вход выполняется по токену, API принимает запросы только со страниц фронтенда благодаря настройке CORS, покупатель видит только свои заказы;

– разделение частей: бэкенд не отдаёт HTML-страниц сайта и отвечает только в формате JSON, а фронтенд получает все данные только через API;

– разделение ролей: учётная запись администратора используется только в панели администрирования и не может применяться для покупок на сайте;

– понятность: все сообщения и подписи интерфейса выводятся на русском языке, а пользователь получает уведомление о результате своих действий.

Сформулированные требования стали основой для проектирования архитектуры и базы данных приложения, которое описано во второй главе.

ГЛАВА 2. ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ ПИЦЦЕРИИ

2.1 Проектирование архитектуры и базы данных

Проект разделён на две части. Папка ksp содержит бэкенд – проект Django с именем ksp и одним приложением ksa. В проекте ksp находятся общие настройки (файл settings.py) и таблица адресов (файл urls.py). Приложение ksa содержит всю логику пиццерии: модели данных, сериализаторы, представления API, настройки панели администратора, миграции, начальные данные меню и автоматические тесты. Папка frontend содержит клиентскую часть: семь HTML-страниц, стили style.css и сценарии JavaScript; она никак не зависит от Django.

Архитектура приложения показана в соответствии с рисунком 2. Браузер сначала загружает с сервера фронтенда страницу, стили и сценарии. Затем сценарий отправляет запросы на сервер бэкенда, где маршрутизатор URL передаёт их программному интерфейсу или панели администратора. Представления API работают с данными через модели ORM, которые сохраняют информацию в базе данных SQLite, а сериализаторы превращают результат в JSON.

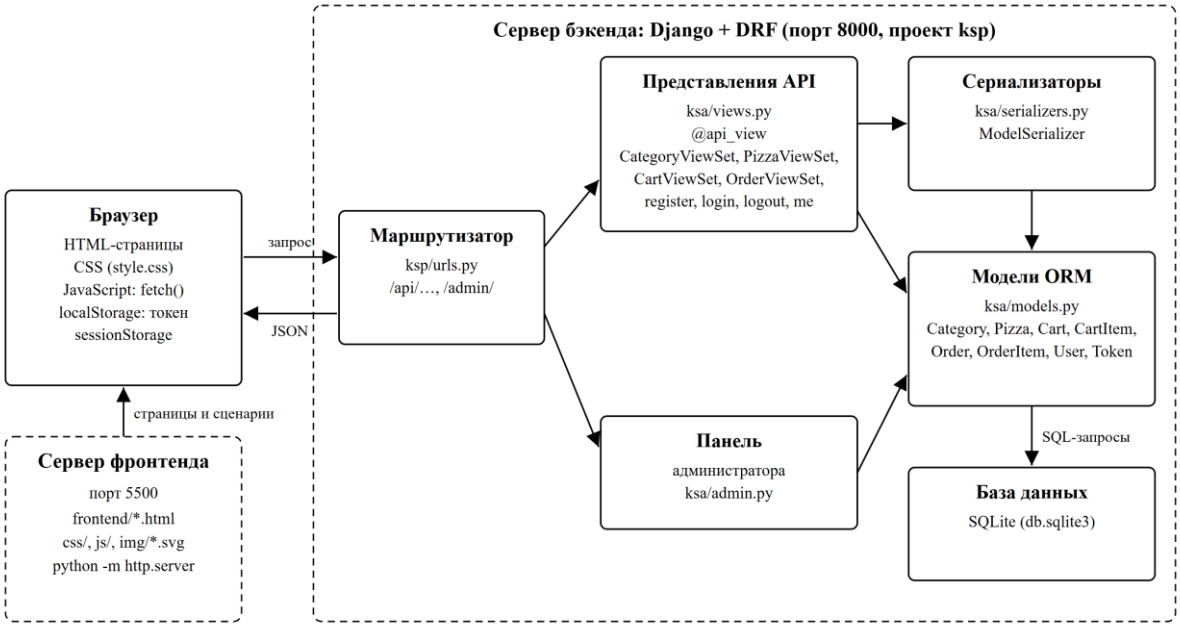


Рисунок 2 – Архитектура веб-приложения пиццерии

Страницы фронтенда содержат только каркас. При открытии страницы её сценарий запрашивает у API нужные данные функцией fetch и собирает из них разметку, а действия с корзиной и оформление заказа отправляются запросами POST в формате JSON. Сценарии страниц заказов периодически запрашивают у программного интерфейса статусы заказов.

На основе понятий предметной области спроектирована структура базы данных. ER-диаграмма приведена в соответствии с рисунком 3. Для хранения пользователей используется встроенная модель User из системы аутентификации Django, остальные шесть моделей созданы в приложении ksa. Для входа по токену дополнительно используется встроенная модель Token библиотеки Django REST Framework: она хранит для каждого пользователя один случайный ключ.

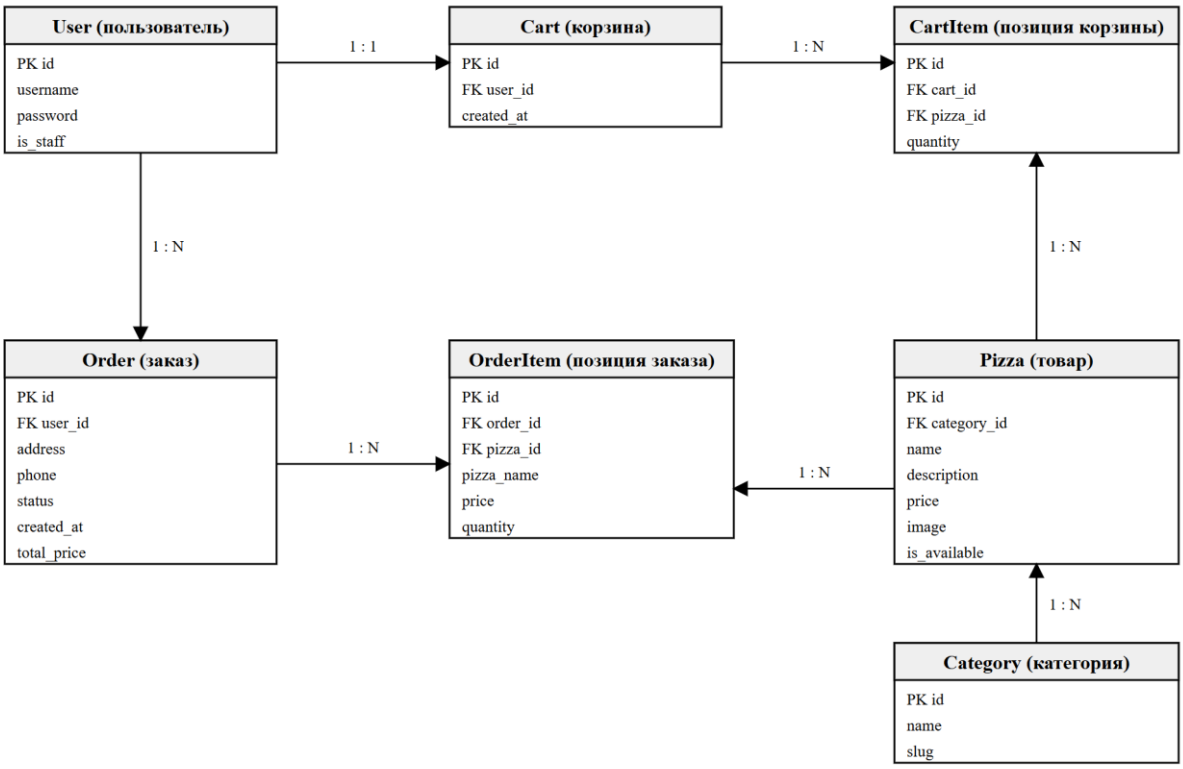


Рисунок 3 – ER-диаграмма базы данных

Назначение таблиц базы данных и их основные поля описаны в таблице 4.

Таблица 4 – Описание сущностей базы данных

Сущность	Основные поля	Назначение
Category	name, slug	Категория меню: пицца, закуски, соусы, напитки; slug используется в адресе фильтра
Pizza	category, name, description, price, image, is_available	Товар меню с ценой, описанием состава и фотографией; признак наличия позволяет скрыть товар
Cart	user, created_at	Корзина пользователя; у каждого пользователя одна корзина (связь «один к одному»)
CartItem	cart, pizza, quantity	Позиция корзины; пара «корзина – товар» уникальна
Order	user, address, phone, status, created_at, total_price	Заказ с контактными данными, статусом и итоговой суммой
OrderItem	order, pizza, pizza_name, price, quantity	Позиция заказа с сохранёнными названием и ценой товара

При проектировании было принято несколько решений, влияющих на надёжность данных. Во-первых, в позиции заказа сохраняются копии названия и цены товара. Если администратор позже изменит цену или удалит

товар из меню, история заказов покупателя останется прежней. По этой же причине связь позиции заказа с товаром настроена так, что при удалении товара ссылка обнуляется, а сама позиция сохраняется. Во-вторых, для пары «корзина – товар» задано ограничение уникальности: повторное добавление товара увеличивает количество в существующей позиции, а не создаёт дубликат. В-третьих, итоговая сумма заказа рассчитывается один раз при оформлении и хранится в поле total_price, а итоговая сумма корзины вычисляется каждый раз заново через свойство модели, потому что корзина постоянно меняется.

Статус заказа хранится в поле status с ограниченным набором значений. Переходы между статусами показаны на диаграмме состояний в соответствии с рисунком 4. Изменение статусов «Новый», «Готовится», «В доставке» и «Завершён» выполняет администратор, а отмена доступна и покупателю, и администратору, но только до передачи заказа курьеру.

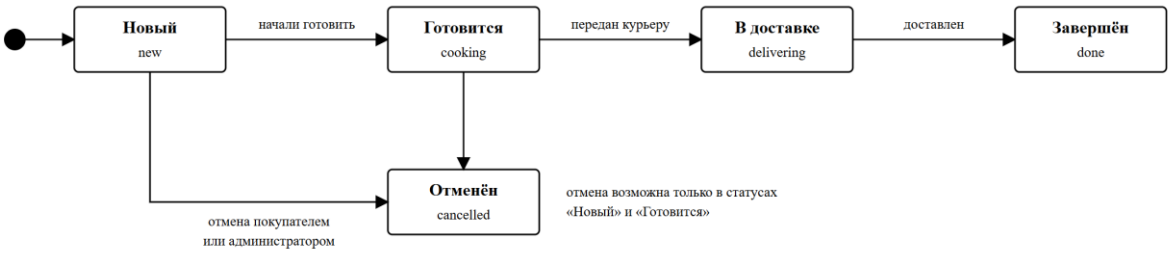


Рисунок 4 – Диаграмма состояний заказа

Таблицы базы данных создаются миграциями Django, которые формируются автоматически по описанию моделей. Для полей и моделей заданы русские названия, поэтому в панели администратора все разделы и поля подписаны на русском языке. Исходное меню – четыре категории и двадцать четыре товара – сохранено в файле начальных данных menu.json и загружается одной командой. Полный листинг моделей данных приведён в приложении 1, представлений – в приложении 2, сериализаторов – в приложении 3.

2.2 Разработка серверной и клиентской частей

Серверная часть состоит только из программного интерфейса. В настройках проекта для ответов оставлен только JSONRenderer, поэтому сервер не отдаёт HTML-страниц сайта, а все страницы находятся в клиентской части. Страницы фронтенда и их сценарии приведены в таблице 5.

Таблица 5 – Страницы клиентской части

Страница	Сценарий	Назначение	Доступ
index.html	menu.js	Меню с разделами и фильтром по категориям, добавление в корзину	все
cart.html	cart.js	Корзина: изменение количества и удаление позиций	покупатель
checkout.html	checkout.js	Оформление заказа	покупатель
orders.html	orders.js	Список заказов с обновлением статусов	покупатель
order.html	order.js	Страница заказа со шкалой статуса и отменой	покупатель
login.html	login.js	Вход по имени пользователя и паролю	все
register.html	register.js	Регистрация нового покупателя	все
все страницы	app.js	Запросы к API, меню в шапке, уведомления, анимации	все

Программный интерфейс построен из четырёх наборов представлений и четырёх функций входа. CategoryViewSet и PizzaViewSet только отдают данные меню, CartViewSet управляет корзиной, а OrderViewSet работает с заказами. Маршрутизатор DefaultRouter автоматически создаёт для них адреса [14]. Функции register, login, logout и me с декоратором api_view отвечают за регистрацию, вход, выход и данные текущего пользователя. Все адреса перечислены в таблице 6.

Таблица 6 – Адреса программного интерфейса

Метод и адрес	Назначение
POST /api/auth/register/	Регистрация, в ответе токен и имя пользователя
POST /api/auth/login/	Вход по имени и паролю, в ответе токен
POST /api/auth/logout/	Выход: удаление токена
GET /api/auth/me/	Имя пользователя, число товаров в корзине, адрес и телефон из последнего заказа
GET /api/categories/	Список категорий меню
GET /api/pizzas/?category=<slug>	Список доступных товаров, при необходимости по категории
GET /api/cart/	Содержимое корзины текущего пользователя
POST /api/cart/add/, /update_item/, /remove/, /clear/	Добавление, изменение количества, удаление позиции и очистка корзины
GET, POST /api/orders/	Список заказов пользователя и оформление нового заказа
GET /api/orders/<id>/, /api/orders/<id>/status/	Заказ и его текущий статус с названием на русском языке
POST /api/orders/<id>/cancel/	Отмена заказа

Оформление заказа вынесено в отдельную функцию make_order, которую вызывает метод create набора OrderViewSet после проверки адреса и телефона. Функция создаёт заказ, переносит в него позиции корзины с текущими ценами, рассчитывает итоговую сумму и очищает корзину. Все эти действия выполняются в одной транзакции базы данных (декоратор transaction.atomic): если на любом шаге произойдёт ошибка, база данных вернётся в исходное состояние и не останется заказа без позиций или корзины, очищенной без заказа [7].

Доступ к корзине и заказам ограничен. Для этого написан класс прав доступа IsCustomer, который пропускает только вошедшего пользователя, не являющегося сотрудником; он подключён к наборам CartViewSet и OrderViewSet. Без токена API отвечает кодом 401, а с токеном сотрудника – кодом 403. Учётная запись администратора используется только в панели администрирования: при попытке войти с ней на сайте API возвращает сообщение о том, что этот аккаунт нельзя использовать на сайте. Все запросы к заказам дополнительно фильтруются по текущему пользователю, поэтому открыть чужой заказ по адресу нельзя – сервер вернёт ошибку 404.

Вход в систему выполняется по токену. При регистрации или входе сервер проверяет имя пользователя и пароль, выдаёт токен и возвращает его

вместе с именем пользователя, как показано в листинге 1. Фронтенд сохраняет токен в localStorage и отправляет его в заголовке Authorization: Token в каждом запросе, а класс TokenAuthentication по этому заголовку определяет пользователя [23]. При выходе токен удаляется из базы данных, поэтому старый ключ перестаёт работать. Так как API не использует вход по cookie, защита от межсайтовой подделки запросов CSRF для него не требуется, а запросы со сторонних сайтов браузер не пропустит из-за настройки CORS, где разрешён только адрес фронтенда. При регистрации пароль проверяется стандартными валидаторами Django: он должен быть не короче восьми символов, не совпадать с именем и не состоять только из цифр.

Листинг 1 – Вход пользователя по токену (файл views.py)

```
def login(request):
    username = request.data.get('username', '')
    password = request.data.get('password', '')
    user = authenticate(username=username, password=password)
    if user is None:
        return Response({'error': 'Неверное имя пользователя или пароль'},
status=status.HTTP_400_BAD_REQUEST)
    if user.is_staff:
        return Response({'error': 'Этот аккаунт нельзя использовать на
сайте'}, status=status.HTTP_403_FORBIDDEN)
    token, _ = Token.objects.get_or_create(user=user)
    return Response({'token': token.key, 'username': user.username})
```

Клиентская часть состоит из семи HTML-страниц: index.html, cart.html, checkout.html, orders.html, order.html, login.html и register.html. Каждая страница содержит одинаковые шапку, подвал и область для уведомлений, а пункты меню в шапке строит общий сценарий app.js: гость видит ссылки «Войти» и «Регистрация», а покупатель – «Мои заказы», «Корзина» со счётчиком товаров и кнопку выхода. Все запросы к серверу выполняет функция api, приведённая в листинге 2: она подставляет адрес бэкенда и токен, отправляет данные в формате JSON и при ответе 401 удаляет устаревший токен. В сценарии меню menu.js товары группируются по категориям, поэтому каждая категория выводится отдельным разделом со своим заголовком.

Листинг 2 – Функция запросов к API (файл app.js)

```
function api(url, method, data) {
    var options = {method: method || 'GET', headers: {}};
    if (getToken()) {
        options.headers['Authorization'] = 'Token ' + getToken();
    }
    if (data !== undefined) {
        options.headers['Content-Type'] = 'application/json';
        options.body = JSON.stringify(data);
    }
    return fetch(API + url, options).then(function (response) {
        if (response.status == 401) {
            clearLogin();
        }
        if (response.status == 204) {
            return {status: 204, data: {}};
        }
        return response.json().then(function (json) {
            return {status: response.status, data: json};
        });
    });
}
```

Взаимодействие с корзиной выполняется без перезагрузки страницы. Когда покупатель нажимает кнопку «В корзину», сценарий menu.js отправляет запрос POST на адрес /api/cart/add/ с номером товара и токеном. Сервер возвращает JSON с новым количеством товара, числом товаров в корзине и итоговой суммой. Последовательность обмена сообщениями показана в соответствии с рисунком 5.

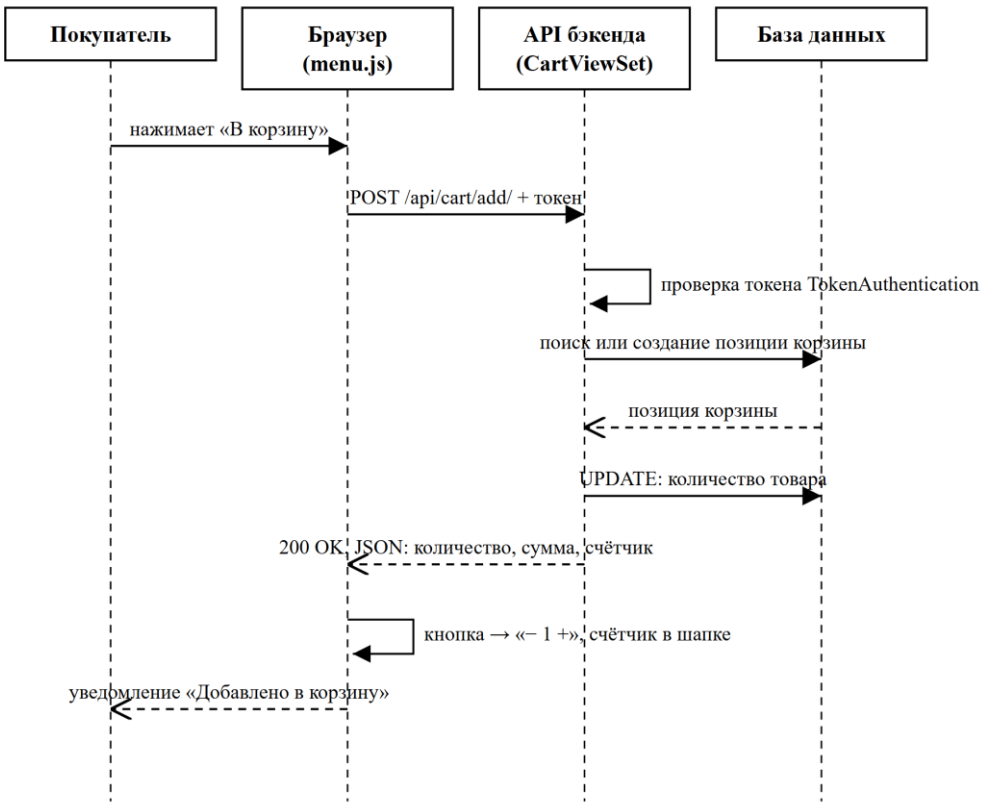


Рисунок 5 – Диаграмма последовательности добавления товара в корзину

Получив ответ, сценарий заменяет кнопку «В корзину» переключателем количества «− 1 +», обновляет счётчик в шапке и показывает всплывающее уведомление. Кнопки «плюс» и «минус» работают так же: при уменьшении количества до нуля позиция удаляется, и в карточке снова появляется кнопка «В корзину». На странице корзины при удалении позиции строка плавно скрывается, а итоговая сумма пересчитывается.

Отслеживание статуса заказа реализовано периодическим опросом сервера. На странице заказа сценарий раз в три секунды запрашивает адрес /api/orders/<id>/status/, а на странице списка заказов – адрес /api/orders/, и сравнивает полученные статусы с отображаемыми на странице. Если статус изменился, сценарий перестраивает только нужные элементы: метку статуса, шкалу этапов и кнопку отмены, после чего показывает уведомление. Чтобы не создавать лишнюю нагрузку на сервер, запросы не отправляются, пока вкладка браузера свёрнута, а когда заказ завершён или отменён, опрос прекращается.

Внешний вид страниц определяется файлом style.css. Цвета, радиусы скругления и тени заданы через CSS-переменные, карточки товаров выстраиваются в сетку, число колонок которой зависит от ширины экрана. Для кнопок, карточек и уведомлений заданы плавные переходы, а если в настройках системы пользователя включено уменьшение движения, анимации отключаются.

Главная страница приложения для гостя приведена в соответствии с рисунком 6. Меню разделено на категории, у каждого товара указаны фотография, название, состав, размер и цена.

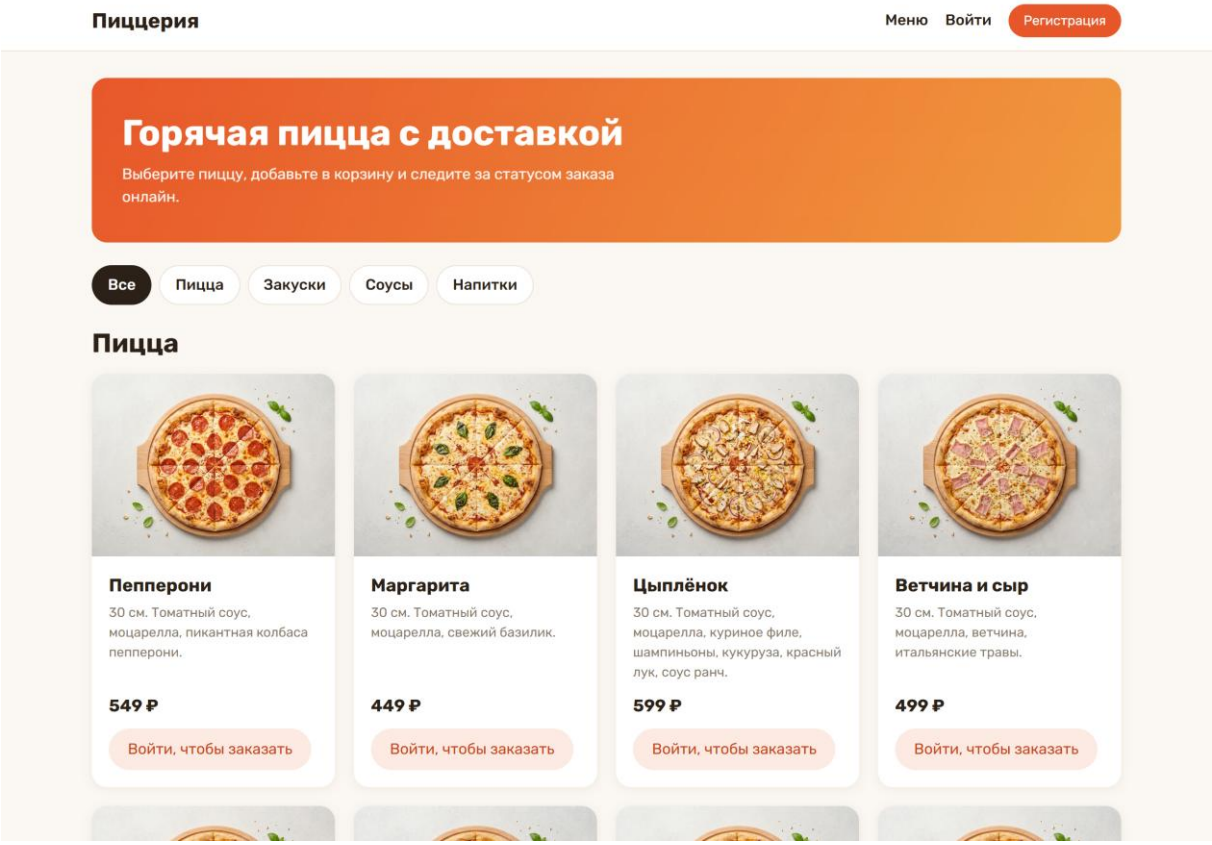


Рисунок 6 – Главная страница с меню

После входа в систему покупатель может добавлять товары в корзину прямо из меню. Состояние карточек после добавления товара показано в соответствии с рисунком 7: у добавленных товаров вместо кнопки отображается переключатель количества, в шапке обновился счётчик корзины, а в правом нижнем углу появилось уведомление.

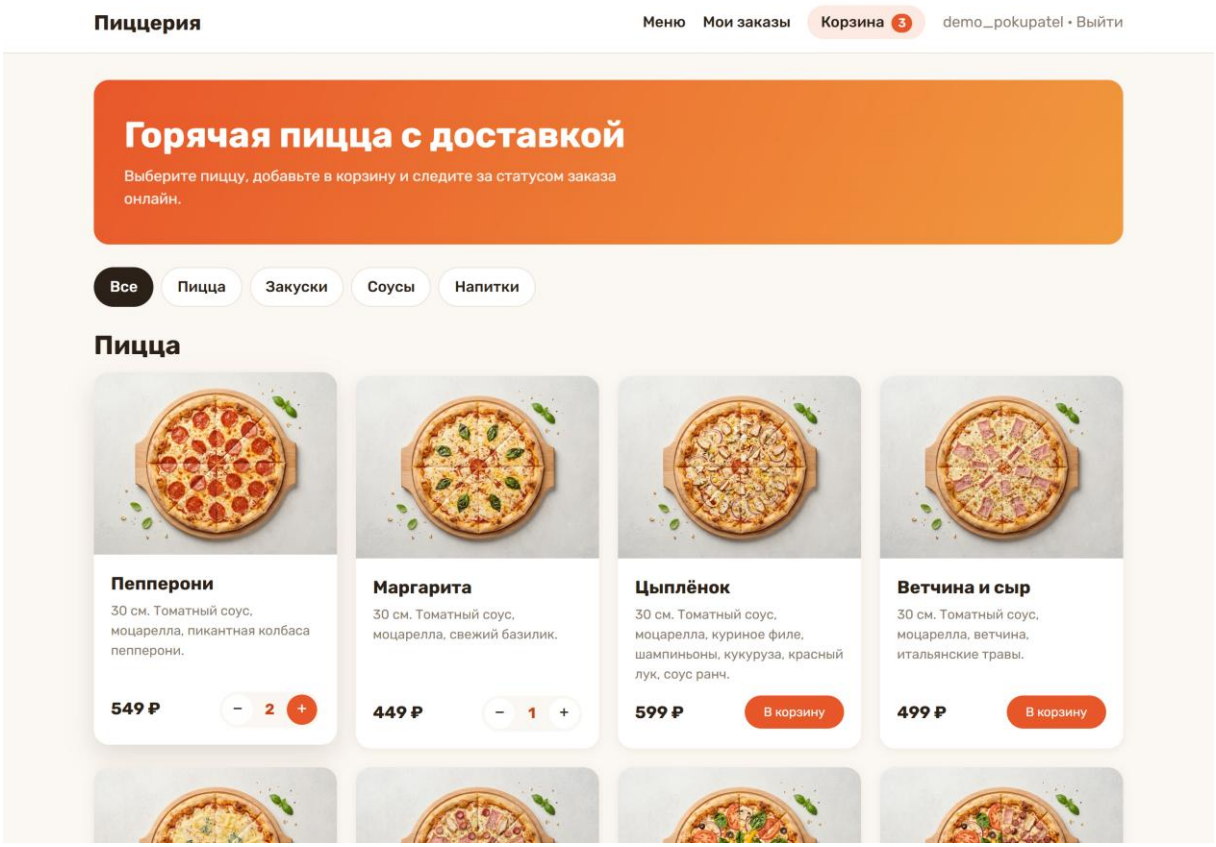


Рисунок 7 – Меню после добавления товара в корзину

Страница корзины приведена в соответствии с рисунком 8. Для каждой позиции указаны цена за штуку, количество и сумма, справа выводится итоговая сумма заказа и кнопка перехода к оформлению.

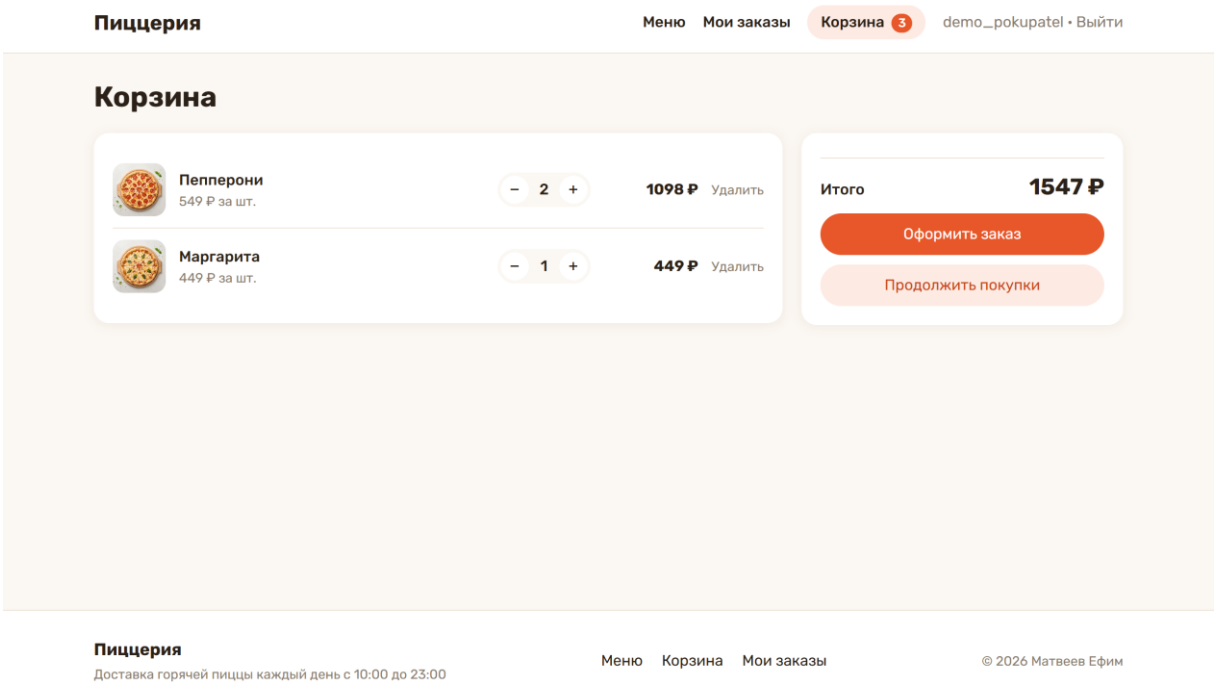


Рисунок 8 – Страница корзины

На странице оформления заказа в соответствии с рисунком 9 покупатель указывает адрес доставки и телефон. Если покупатель уже делал заказы, поля заполняются данными из предыдущего заказа. Перед сохранением номер телефона проверяется: он должен содержать от 10 до 15 цифр.

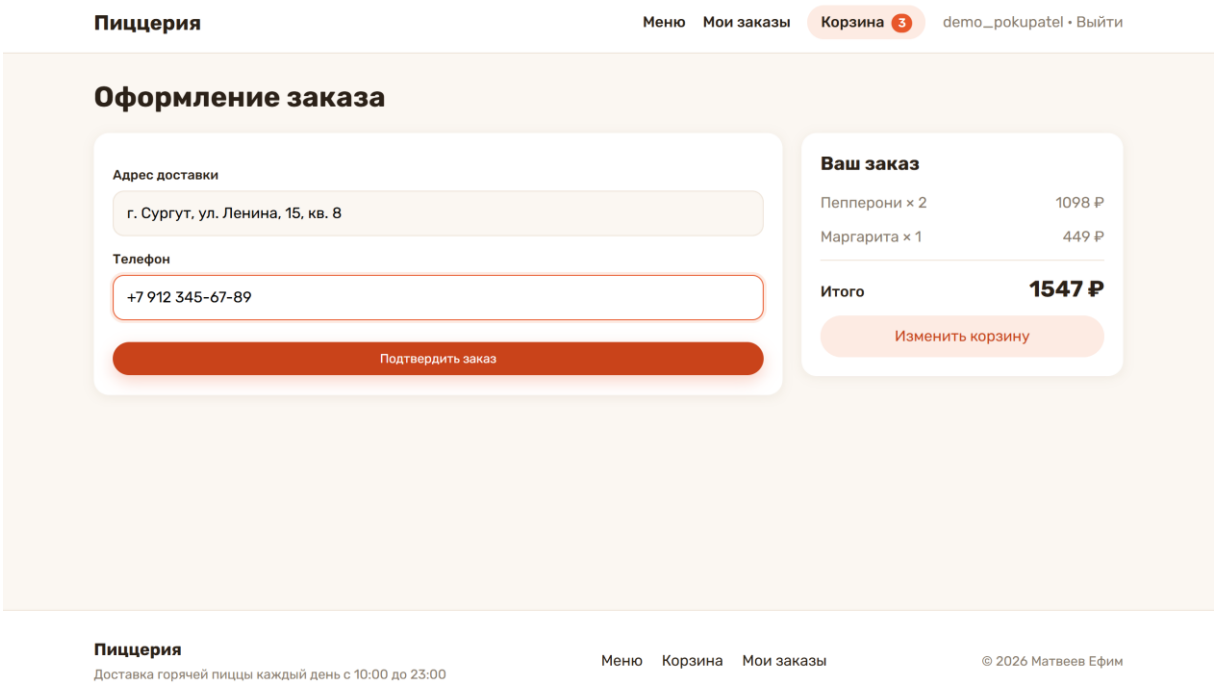


Рисунок 9 – Страница оформления заказа

После оформления открывается страница заказа, приведённая в соответствии с рисунком 10. На шкале отмечены пройденные этапы, текущий этап выделен. Здесь же выводятся состав заказа, адрес и кнопка отмены, если заказ ещё можно отменить.

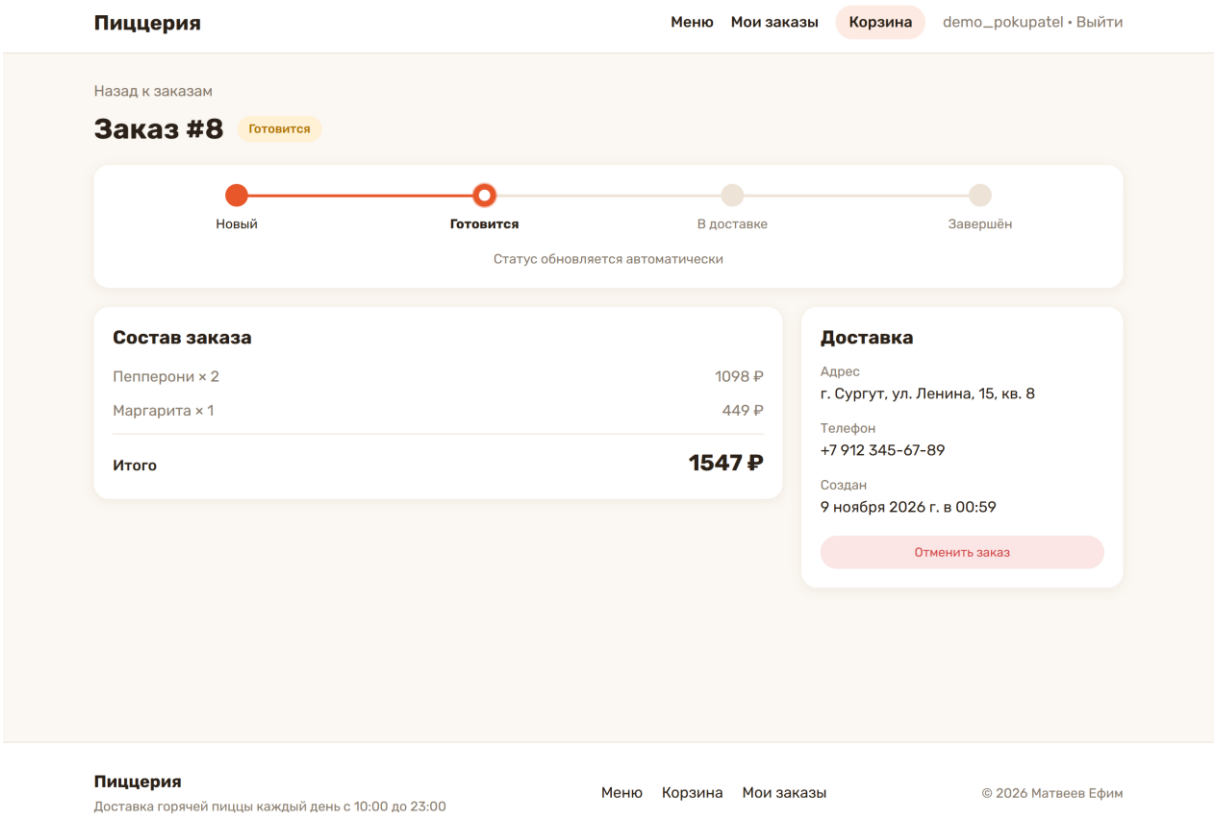


Рисунок 10 – Страница заказа с отслеживанием статуса

Администратор изменяет статус заказа в панели администрирования в соответствии с рисунком 11. Сразу после сохранения изменения статус обновляется на странице заказа у покупателя.

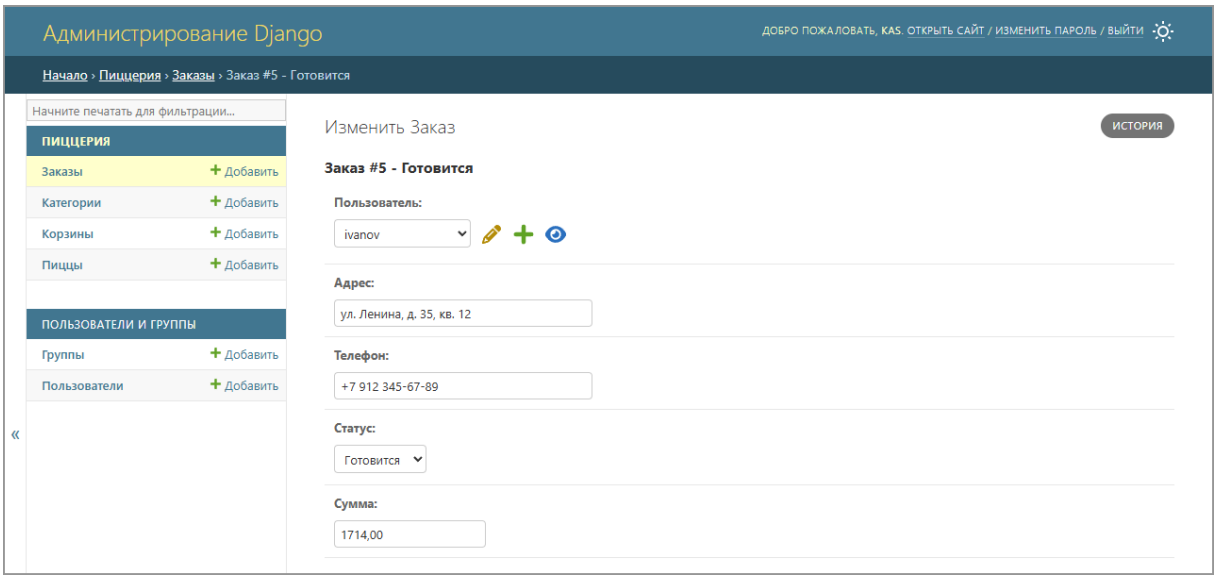


Рисунок 11 – Изменение статуса заказа в панели администратора

Вход в систему приведён в соответствии с рисунком 12. Если имя пользователя или пароль неверны, сервер отвечает кодом 400, и над формой выводится сообщение об ошибке. После успешного входа покупатель возвращается на страницу, с которой его перенаправили на вход, например в корзину.

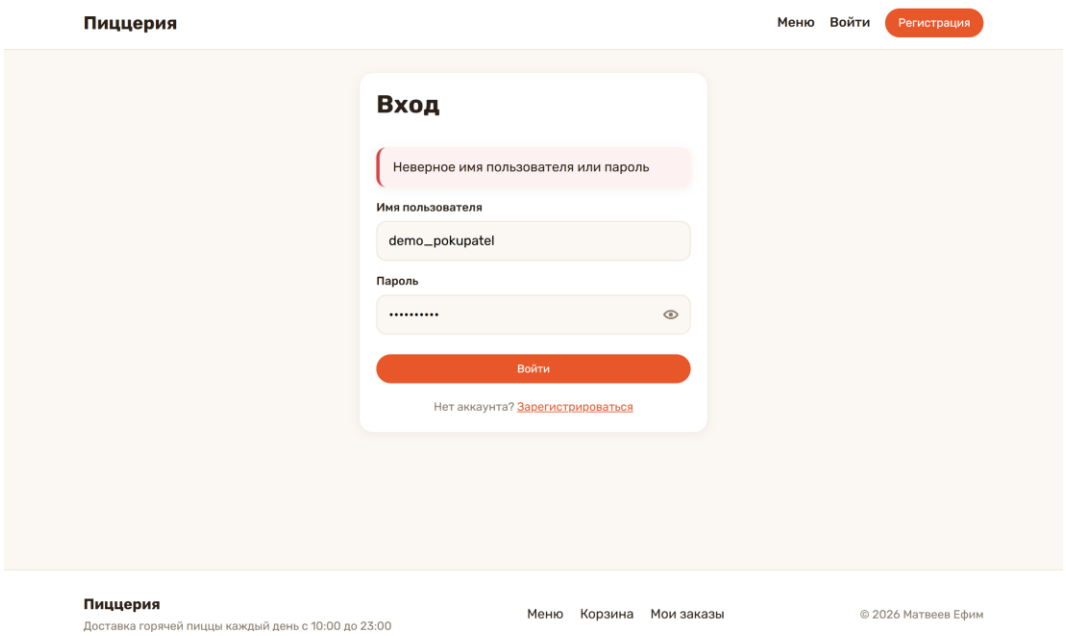


Рисунок 12 – Страница входа с сообщением об ошибке

Все данные страницы получают от бэкенда. Пример ответа API на запрос заказа приведён в соответствии с рисунком 13: сервер возвращает объект JSON с адресом, статусом, итоговой суммой и позициями заказа, а запрос выполняется только с токеном покупателя.

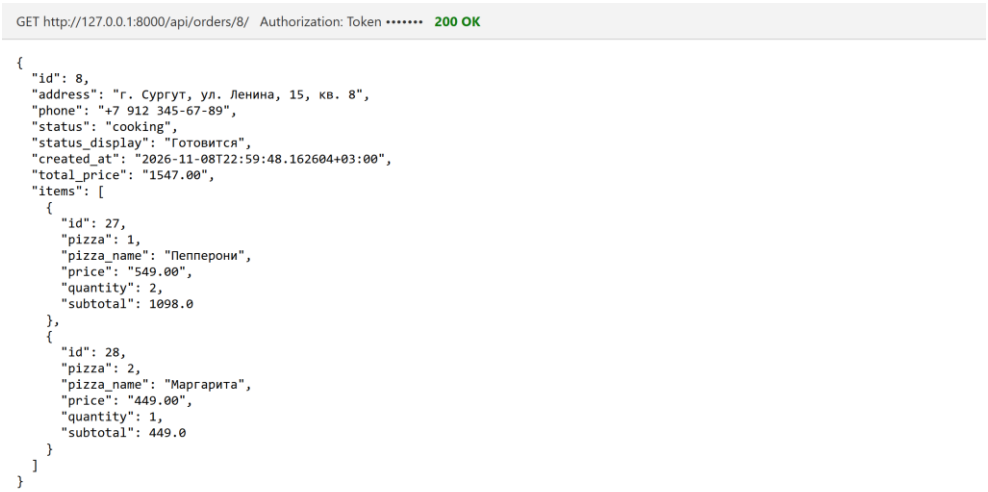


Рисунок 13 – Ответ API на запрос GET /api/orders/<id>/

2.3 Тестирование веб-приложения

Тестирование проводилось в два этапа: автоматическое модульное тестирование программного интерфейса и ручная проверка работы клиентской части в браузере при одновременно запущенных серверах фронтенда и бэкенда.

Автоматические тесты написаны с использованием класса APITestCase библиотеки Django REST Framework. Для каждого запуска Django создаёт отдельную тестовую базу данных, поэтому тесты не влияют на рабочие данные. Тестовый клиент APIClient отправляет запросы к API в формате JSON, а метод credentials подставляет токен покупателя в заголовок Authorization. Перечень тестов и результаты их выполнения приведены в таблице 7.

Таблица 7 – Результаты автоматического тестирования

№	Проверяемая функция	Результат
1	API отвечает в формате JSON	пройден
2	Меню фильтруется по категории	пройден
3	Скрытый товар не попадает в меню	пройден
4	Корзина и заказы недоступны без токена (код 401)	пройден
5	Регистрация выдаёт токен, по нему открываются данные пользователя	пройден
6	Регистрация с занятым именем и разными паролями отклоняется	пройден
7	Слишком простой пароль не принимается	пройден
8	Вход с неверным и верным паролем, выход удаляет токен	пройден
9	Учётная запись администратора не используется на сайте	пройден
10	Добавление, изменение количества и удаление позиций корзины	пройден
11	Количество 0 удаляет позицию из корзины	пройден
12	Оформление заказа создаёт заказ и очищает корзину	пройден
13	Отклонение пустого адреса и некорректного телефона	пройден
14	Пустую корзину нельзя оформить	пройден
15	Статус заказа и отмена заказа	пройден
16	Заказ в доставке нельзя отменить	пройден
17	Покупатель не может открыть чужой заказ	пройден
18	Адрес из последнего заказа подставляется при оформлении	пройден
19	Панель администратора работает	пройден
20	Сервер не отдаёт HTML-страниц сайта	пройден

Тесты запускаются из папки ksp командой python manage.py test ksa. Все 20 тестов выполнены успешно. Особое внимание при тестировании уделено безопасности: проверено, что корзина и заказы недоступны без токена, что один покупатель не может просмотреть заказ другого, что после выхода токен перестаёт работать и что учётная запись администратора не может быть использована для покупок на сайте.

При ручном тестировании в браузерах Google Chrome и Microsoft Edge проверялись сценарии, которые сложно покрыть автоматическими тестами:

– добавление товаров из меню и изменение их количества переключателем «− N +» без перезагрузки страницы;

– появление и автоматическое скрытие уведомлений;

– плавное удаление строк в корзине и пересчёт итоговой суммы;

– переключение категорий меню без перезагрузки страницы;

– обновление статуса заказа на странице покупателя после изменения статуса в панели администратора;

– регистрация, вход и выход, а также переход на страницу входа при открытии корзины без входа;

– отображение интерфейса на экране мобильного телефона.

Для проверки отслеживания статуса заказ последовательно переводился в панели администратора из статуса «Новый» в статусы «Готовится», «В доставке» и «Завершён». Во всех случаях изменение отображалось на открытой странице заказа в течение трёх секунд без перезагрузки: менялась метка статуса, шкала этапов перестраивалась, а после перехода в статус «В доставке» исчезала кнопка отмены. Отображение главной страницы на экране мобильного телефона показано в соответствии с рисунком 14.

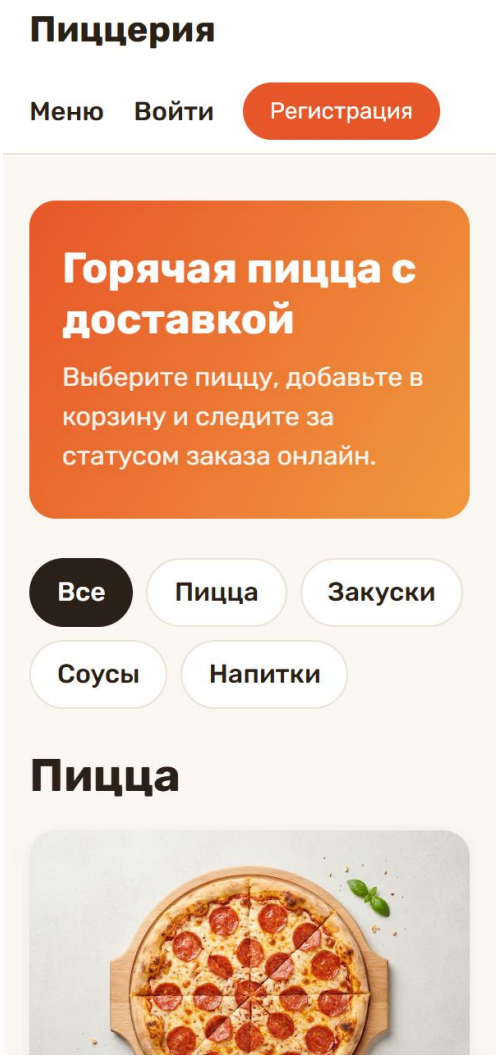


Рисунок 14 – Главная страница на экране мобильного телефона

Ошибок, препятствующих работе приложения, по результатам тестирования не выявлено. Для развёртывания бэкенда достаточно установить зависимости из файла requirements.txt, выполнить миграции, загрузить начальное меню командой python manage.py loaddata menu, создать учётную запись администратора и запустить сервер командой python manage.py runserver 8000. Фронтенд запускается отдельно из папки frontend командой python -m http.server 5500. Исходный код приложения размещён в репозитории на GitHub [13].

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы было спроектировано и разработано веб-приложение пиццерии, которое позволяет просматривать меню, формировать корзину, оформлять заказ и отслеживать его статус. Поставленная цель достигнута, все задачи решены.

В результате анализа предметной области рассмотрен процесс приёма заказа в пиццерии и выявлены проблемы приёма заказов по телефону. Сравнение агрегаторов доставки, приложений крупных сетей и сайтов на конструкторах показало, что собственное веб-приложение позволяет избежать комиссии посредников и сохранить контроль над данными покупателей, не отказываясь от функции отслеживания заказа.

Рассмотрены технологии разработки веб-приложений и принципы архитектурного стиля REST, обоснован выбор фреймворка Django, библиотеки Django REST Framework с аутентификацией по токену, СУБД SQLite и стандартных средств HTML, CSS и JavaScript. Сформулированы функциональные требования для трёх ролей пользователей и нефункциональные требования к адаптивности, отзывчивости, безопасности, разделению частей и актуальности статуса заказа.

Спроектированы архитектура приложения из двух серверов и база данных из семи таблиц, построены диаграммы вариантов использования, состояний заказа и последовательности добавления товара в корзину. Разработан REST API, который отвечает только в формате JSON, и клиентская часть, которая получает все данные через API: корзина изменяется без перезагрузки страницы, покупатель регистрируется и входит по токену, получает уведомления о своих действиях, а статус заказа обновляется на экране автоматически после изменения его администратором.

Приложение проверено двадцатью автоматическими тестами API и вручную в браузере. Все тесты выполнены успешно, ошибок,

препятствующих работе, не выявлено. Тем самым подтверждена гипотеза работы: покупатель может самостоятельно оформить заказ и видеть, на каком этапе он находится, не обращаясь к сотрудникам пиццерии.

Практическим результатом работы стало работающее веб-приложение из двух частей: программный интерфейс с меню из четырёх категорий и двадцати четырёх товаров, корзиной, заказами и входом по токену, и клиентская часть со страницами меню, корзины, оформления заказа и отслеживания статуса. Исходный код размещён в открытом репозитории вместе с инструкцией по запуску и файлом начальных данных, поэтому приложение можно развернуть на другом компьютере за несколько минут.

Полнота решения задач оценивается как достаточная для использования приложения небольшой пиццерией. Вместе с тем приложение можно развивать в следующих направлениях:

– подключение онлайн-оплаты заказа;

– передача статуса заказа через технологию WebSocket вместо периодического опроса сервера;

– учёт времени доставки и назначение заказов курьерам;

– переход на СУБД PostgreSQL при росте количества заказов.

– мобильное приложение, которое будет работать с тем же REST API.

Результаты курсовой работы рекомендуется использовать при создании сайта небольшого заведения общественного питания, а также в качестве учебного примера разработки веб-приложения на фреймворке Django.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Федеральный закон от 27.07.2006 № 152-ФЗ «О персональных данных» // Собрание законодательства Российской Федерации. – 2006. – № 31 (ч. 1). – Ст. 3451.

2. Федеральный закон от 27.07.2006 № 149-ФЗ «Об информации, информационных технологиях и о защите информации» // Собрание законодательства Российской Федерации. – 2006. – № 31 (ч. 1). – Ст. 3448.

3. Закон Российской Федерации от 07.02.1992 № 2300-1 «О защите прав потребителей» // Собрание законодательства Российской Федерации. – 1996. – № 3. – Ст. 140.

4. ГОСТ 34.602-2020. Информационные технологии. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы : межгосударственный стандарт : введен впервые 2022-01-01. – Москва : Российский институт стандартизации, 2022.

5. ГОСТ 7.32-2017. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления : межгосударственный стандарт : введен 2018-07-01. – Москва : Стандартинформ, 2017.

6. ГОСТ Р 7.0.100-2018. Система стандартов по информации, библиотечному и издательскому делу. Библиографическая запись. Библиографическое описание. Общие требования и правила составления : национальный стандарт Российской Федерации : введен 2019-07-01. – Москва : Стандартинформ, 2018.

7. Дронов, В. А. Django 4. Практика создания веб-сайтов на Python / В. А. Дронов. – Санкт-Петербург : БХВ-Петербург, 2023. – 800 с.

8. Лутц, М. Изучаем Python. Том 1 / М. Лутц ; перевод с английского. – 5-е изд. – Санкт-Петербург : Диалектика, 2019. – 832 с.

9. Тузовский, А. Ф. Проектирование и разработка web-приложений : учебное пособие для академического бакалавриата / А. Ф. Тузовский. – Москва : Юрайт, 2019. – 218 с. – (Университеты России).

10. Флэнаган, Д. JavaScript. Полное руководство / Д. Флэнаган ; перевод с английского. – 7-е изд. – Санкт-Петербург : Диалектика, 2021. – 720 с.

11. Фримен, Э. Изучаем программирование на JavaScript / Э. Фримен, Э. Робсон ; перевод с английского. – Санкт-Петербург : Питер, 2017. – 640 с.

12. Melé, A. Django 4 By Example : Build powerful and reliable Python web applications from scratch / A. Melé. – 4th ed. – Birmingham : Packt Publishing, 2022. – 766 p.

13. Матвеев, Е. К. Веб-приложение пиццерии kursach-pizza : исходный код // GitHub. – URL: https://github.com/kastrulin75/kursach-pizza (дата обращения: 01.11.2026).

14. Django REST framework : официальный сайт. – URL: https://www.django-rest-framework.org/ (дата обращения: 20.10.2026).

15. Django documentation : официальная документация фреймворка Django. – URL: https://docs.djangoproject.com/en/6.1/ (дата обращения: 20.10.2026).

16. Fetch API // MDN Web Docs. – URL: https://developer.mozilla.org/ru/docs/Web/API/Fetch_API (дата обращения: 22.10.2026).

17. Intersection Observer API // MDN Web Docs. – URL: https://developer.mozilla.org/ru/docs/Web/API/Intersection_Observer_API (дата обращения: 22.10.2026).

18. Python documentation : официальная документация языка Python. – URL: https://docs.python.org/3/ (дата обращения: 20.10.2026).

19. SQLite : официальный сайт. – URL: https://www.sqlite.org/ (дата обращения: 20.10.2026).

20. django-cors-headers : описание пакета // Python Package Index. – URL: https://pypi.org/project/django-cors-headers/ (дата обращения: 09.11.2026).

21. Fielding, R. T. Architectural Styles and the Design of Network-based Software Architectures : dissertation / R. T. Fielding. – Irvine : University of California, 2000. – 162 p. – URL: https://ics.uci.edu/~fielding/pubs/dissertation/top.htm (дата обращения: 09.11.2026).

22. Bray, T. The JavaScript Object Notation (JSON) Data Interchange Format : RFC 8259 / T. Bray. – Internet Engineering Task Force, 2017. – URL: https://www.rfc-editor.org/rfc/rfc8259 (дата обращения: 09.11.2026).

23. Authentication // Django REST framework. – URL: https://www.django-rest-framework.org/api-guide/authentication/ (дата обращения: 09.11.2026).

24. Web Storage API // MDN Web Docs. – URL: https://developer.mozilla.org/ru/docs/Web/API/Web_Storage_API (дата обращения: 09.11.2026).

ПРИЛОЖЕНИЕ 1

Листинг модели данных (файл ksp/ksa/models.py)

```
from django.db import models
from django.contrib.auth.models import User


class Category(models.Model):
    name = models.CharField('Название', max_length=100)
    slug = models.SlugField('Адрес (slug)', unique=True)

    class Meta:
        verbose_name = 'Категория'
        verbose_name_plural = 'Категории'

    def __str__(self):
        return self.name


class Pizza(models.Model):
    category = models.ForeignKey(Category, on_delete=models.CASCADE,
related_name='pizzas', verbose_name='Категория')
    name = models.CharField('Название', max_length=150)
    description = models.TextField('Описание', blank=True)
    price = models.DecimalField('Цена', max_digits=8, decimal_places=2)
    image = models.ImageField('Фото', upload_to='pizzas/', blank=True,
null=True)
    is_available = models.BooleanField('В наличии', default=True)

    class Meta:
        verbose_name = 'Пицца'
        verbose_name_plural = 'Пиццы'

    def __str__(self):
        return self.name


class Cart(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE,
related_name='cart', verbose_name='Пользователь')
    created_at = models.DateTimeField('Создана', auto_now_add=True)

    class Meta:
        verbose_name = 'Корзина'
        verbose_name_plural = 'Корзины'

    def __str__(self):
        return f"Корзина {self.user.username}"

    @property
    def total_price(self):
        return sum(item.subtotal for item in self.items.all())

    @property
    def items_count(self):
        return sum(item.quantity for item in self.items.all())
```

```
class CartItem(models.Model):
    cart = models.ForeignKey(Cart, on_delete=models.CASCADE,
related_name='items', verbose_name='Корзина')
    pizza = models.ForeignKey(Pizza, on_delete=models.CASCADE,
verbose_name='Пицца')
    quantity = models.PositiveIntegerField('Количество', default=1)

    class Meta:
        unique_together = ('cart', 'pizza')
        verbose_name = 'Позиция корзины'
        verbose_name_plural = 'Позиции корзины'

    @property
    def subtotal(self):
        return self.pizza.price * self.quantity


class Order(models.Model):
    STATUS_CHOICES = [
        ('new', 'Новый'),
        ('cooking', 'Готовится'),
        ('delivering', 'В доставке'),
        ('done', 'Завершён'),
        ('cancelled', 'Отменён'),
    ]

    user = models.ForeignKey(User, on_delete=models.CASCADE,
related_name='orders', verbose_name='Пользователь')
    address = models.CharField('Адрес', max_length=255)
    phone = models.CharField('Телефон', max_length=20)
    status = models.CharField('Статус', max_length=20,
choices=STATUS_CHOICES, default='new')
    created_at = models.DateTimeField('Создан', auto_now_add=True)
    total_price = models.DecimalField('Сумма', max_digits=10,
decimal_places=2, default=0)

    class Meta:
        verbose_name = 'Заказ'
        verbose_name_plural = 'Заказы'

    def __str__(self):
        return f"Заказ #{self.id} - {self.get_status_display()}"


class OrderItem(models.Model):
    order = models.ForeignKey(Order, on_delete=models.CASCADE,
related_name='items', verbose_name='Заказ')
    pizza = models.ForeignKey(Pizza, on_delete=models.SET_NULL, null=True,
verbose_name='Пицца')
    pizza_name = models.CharField('Название пиццы', max_length=150)
    price = models.DecimalField('Цена', max_digits=8, decimal_places=2)
    quantity = models.PositiveIntegerField('Количество', default=1)

    class Meta:
        verbose_name = 'Позиция заказа'
        verbose_name_plural = 'Позиции заказа'

    @property
    def subtotal(self):
        return self.price * self.quantity
```

ПРИЛОЖЕНИЕ 2

Листинг представлений API (файл ksp/ksa/views.py)

```
import re

from django.contrib.auth import authenticate
from django.db import transaction
from rest_framework import viewsets, status
from rest_framework.authtoken.models import Token
from rest_framework.decorators import action, api_view, permission_classes
from rest_framework.permissions import IsAuthenticated, AllowAny,
BasePermission
from rest_framework.response import Response

from .models import Category, Pizza, Cart, CartItem, Order, OrderItem
from .serializers import (
    CategorySerializer, PizzaSerializer, CartSerializer,
    CartItemSerializer, OrderSerializer, OrderStatusSerializer,
RegisterSerializer
)


class IsCustomer(BasePermission):
    message = 'Войдите в аккаунт покупателя'

    def has_permission(self, request, view):
        return request.user.is_authenticated and not request.user.is_staff


def error_list(errors):
    result = []
    for value in errors.values():
        for text in value:
            result.append(str(text))
    return result


class CategoryViewSet(viewsets.ReadOnlyModelViewSet):
    queryset = Category.objects.all()
    serializer_class = CategorySerializer
    permission_classes = [AllowAny]


class PizzaViewSet(viewsets.ReadOnlyModelViewSet):
    queryset =
Pizza.objects.filter(is_available=True).select_related('category').order_by('
category__id', 'id')
    serializer_class = PizzaSerializer
    permission_classes = [AllowAny]

    def get_queryset(self):
        queryset = super().get_queryset()
        category = self.request.query_params.get('category')
        if category:
            queryset = queryset.filter(category__slug=category)
        return queryset
```

```
def cart_info(cart, item_id, quantity, subtotal=0):
    return {
        'item_id': item_id,
        'quantity': quantity,
        'subtotal': float(subtotal),
        'total': float(cart.total_price),
        'count': cart.items_count,
    }


class CartViewSet(viewsets.ViewSet):
    permission_classes = [IsCustomer]

    def _get_cart(self):
        cart, _ = Cart.objects.get_or_create(user=self.request.user)
        return cart

    def list(self, request):
        serializer = CartSerializer(self._get_cart(), context={'request':
request})
        return Response(serializer.data)

    @action(detail=False, methods=['post'])
    def add(self, request):
        cart = self._get_cart()
        serializer = CartItemSerializer(data=request.data)
        if not serializer.is_valid():
            return Response({'errors': error_list(serializer.errors)},
status=status.HTTP_400_BAD_REQUEST)
        pizza = serializer.validated_data['pizza']
        quantity = serializer.validated_data.get('quantity', 1)

        item, created = CartItem.objects.get_or_create(cart=cart,
pizza=pizza)
        if not created:
            item.quantity = min(item.quantity + quantity, 99)
        else:
            item.quantity = min(quantity, 99)
        item.save()
        return Response(cart_info(cart, item.id, item.quantity,
item.subtotal), status=status.HTTP_200_OK)

    @action(detail=False, methods=['post'])
    def update_item(self, request):
        cart = self._get_cart()
        try:
            item = cart.items.get(id=request.data.get('item_id'))
            quantity = int(request.data.get('quantity', 1))
        except (CartItem.DoesNotExist, ValueError, TypeError):
            return Response({'error': 'Товар не найден'},
status=status.HTTP_404_NOT_FOUND)
        if quantity > 0:
            item.quantity = min(quantity, 99)
            item.save()
            return Response(cart_info(cart, item.id, item.quantity,
item.subtotal))
        item_id = item.id
        item.delete()
        return Response(cart_info(cart, item_id, 0))

    @action(detail=False, methods=['post'])
    def remove(self, request):
        cart = self._get_cart()
```

```
        item_id = request.data.get('item_id')
        try:
            item = cart.items.get(id=item_id)
        except (CartItem.DoesNotExist, ValueError, TypeError):
            return Response({'error': 'Товар не найден'},
status=status.HTTP_404_NOT_FOUND)
        item.delete()
        return Response(cart_info(cart, int(item_id), 0))

    @action(detail=False, methods=['post'])
    def clear(self, request):
        self._get_cart().items.all().delete()
        return Response(status=status.HTTP_204_NO_CONTENT)


class OrderViewSet(viewsets.ModelViewSet):
    serializer_class = OrderSerializer
    permission_classes = [IsCustomer]
    http_method_names = ['get', 'post']

    def get_queryset(self):
        return Order.objects.filter(user=self.request.user).order_by('-
created_at')

    def create(self, request):
        cart = Cart.objects.filter(user=request.user).first()
        if not cart or not cart.items.exists():
            return Response({'errors': ['Корзина пуста']},
status=status.HTTP_400_BAD_REQUEST)

        address = str(request.data.get('address', '')).strip()
        phone = str(request.data.get('phone', '')).strip()
        errors = []
        if not address:
            errors.append('Укажите адрес доставки')
        digits = re.sub(r'\D', '', phone)
        if len(digits) < 10 or len(digits) > 15:
            errors.append('Введите корректный номер телефона')
        if errors:
            return Response({'errors': errors},
status=status.HTTP_400_BAD_REQUEST)

        order = make_order(cart, address, phone)
        return Response(OrderSerializer(order).data,
status=status.HTTP_201_CREATED)

    @action(detail=True, methods=['get'])
    def status(self, request, pk=None):
        order = self.get_object()
        return Response(OrderStatusSerializer(order).data)

    @action(detail=True, methods=['post'])
    def cancel(self, request, pk=None):
        order = self.get_object()
        if order.status not in ('new', 'cooking'):
            return Response({'error': 'Этот заказ уже нельзя отменить'},
status=status.HTTP_400_BAD_REQUEST)
        order.status = 'cancelled'
        order.save()
        return Response(OrderStatusSerializer(order).data)


@transaction.atomic
def make_order(cart, address, phone):
```

```
    order = Order.objects.create(user=cart.user, address=address,
phone=phone)

    total = 0
    for item in cart.items.all():
        OrderItem.objects.create(
            order=order,
            pizza=item.pizza,
            pizza_name=item.pizza.name,
            price=item.pizza.price,
            quantity=item.quantity
        )
        total += item.subtotal

    order.total_price = total
    order.save()
    cart.items.all().delete()
    return order


@api_view(['POST'])
@permission_classes([AllowAny])
def register(request):
    serializer = RegisterSerializer(data=request.data)
    if not serializer.is_valid():
        return Response({'errors': error_list(serializer.errors)},
status=status.HTTP_400_BAD_REQUEST)
    user = serializer.save()
    token = Token.objects.create(user=user)
    return Response({'token': token.key, 'username': user.username},
status=status.HTTP_201_CREATED)


@api_view(['POST'])
@permission_classes([AllowAny])
def login(request):
    username = request.data.get('username', '')
    password = request.data.get('password', '')
    user = authenticate(username=username, password=password)
    if user is None:
        return Response({'error': 'Неверное имя пользователя или пароль'},
status=status.HTTP_400_BAD_REQUEST)
    if user.is_staff:
        return Response({'error': 'Этот аккаунт нельзя использовать на
сайте'}, status=status.HTTP_403_FORBIDDEN)
    token, _ = Token.objects.get_or_create(user=user)
    return Response({'token': token.key, 'username': user.username})


@api_view(['POST'])
@permission_classes([IsAuthenticated])
def logout(request):
    Token.objects.filter(user=request.user).delete()
    return Response(status=status.HTTP_204_NO_CONTENT)


@api_view(['GET'])
@permission_classes([IsCustomer])
def me(request):
    cart, _ = Cart.objects.get_or_create(user=request.user)
    last_order = Order.objects.filter(user=request.user).order_by('-
created_at').first()
    return Response({
        'username': request.user.username,
```

```
    'cart_count': cart.items_count,
    'last_address': last_order.address if last_order else '',
    'last_phone': last_order.phone if last_order else '',
})
```

ПРИЛОЖЕНИЕ 3

Листинг сериализаторов (файл ksp/ksa/serializers.py)

```
from django.contrib.auth.models import User
from django.contrib.auth.password_validation import validate_password
from django.core.exceptions import ValidationError as DjangoValidationError
from rest_framework import serializers
from .models import Category, Pizza, Cart, CartItem, Order, OrderItem


class CategorySerializer(serializers.ModelSerializer):
    class Meta:
        model = Category
        fields = ['id', 'name', 'slug']


class PizzaSerializer(serializers.ModelSerializer):
    category = CategorySerializer(read_only=True)
    category_id = serializers.PrimaryKeyRelatedField(
        queryset=Category.objects.all(), source='category', write_only=True
    )

    class Meta:
        model = Pizza
        fields = ['id', 'category', 'category_id', 'name', 'description',
'price', 'image', 'is_available']


class CartItemSerializer(serializers.ModelSerializer):
    pizza = PizzaSerializer(read_only=True)
    pizza_id = serializers.PrimaryKeyRelatedField(
        queryset=Pizza.objects.filter(is_available=True), source='pizza',
write_only=True
    )
    subtotal = serializers.ReadOnlyField()

    class Meta:
        model = CartItem
        fields = ['id', 'pizza', 'pizza_id', 'quantity', 'subtotal']


class CartSerializer(serializers.ModelSerializer):
    items = CartItemSerializer(many=True, read_only=True)
    total_price = serializers.ReadOnlyField()
    items_count = serializers.ReadOnlyField()

    class Meta:
        model = Cart
        fields = ['id', 'items', 'total_price', 'items_count']


class OrderItemSerializer(serializers.ModelSerializer):
    subtotal = serializers.ReadOnlyField()

    class Meta:
        model = OrderItem
```

```
        fields = ['id', 'pizza', 'pizza_name', 'price', 'quantity',
'subtotal']


class OrderSerializer(serializers.ModelSerializer):
    items = OrderItemSerializer(many=True, read_only=True)
    status_display = serializers.CharField(source='get_status_display',
read_only=True)

    class Meta:
        model = Order
        fields = ['id', 'address', 'phone', 'status', 'status_display',
'created_at', 'total_price', 'items']
        read_only_fields = ['status', 'created_at', 'total_price']


class OrderStatusSerializer(serializers.ModelSerializer):
    status_display = serializers.CharField(source='get_status_display',
read_only=True)

    class Meta:
        model = Order
        fields = ['id', 'status', 'status_display']


class RegisterSerializer(serializers.Serializer):
    username = serializers.CharField(max_length=150, error_messages={'blank':
'Введите имя пользователя', 'required': 'Введите имя пользователя'})
    password1 = serializers.CharField(error_messages={'blank': 'Введите
пароль', 'required': 'Введите пароль'})
    password2 = serializers.CharField(error_messages={'blank': 'Повторите
пароль', 'required': 'Повторите пароль'})

    def validate_username(self, value):
        if User.objects.filter(username__iexact=value).exists():
            raise serializers.ValidationError('Пользователь с таким именем
уже существует')
        return value

    def validate(self, data):
        if data['password1'] != data['password2']:
            raise serializers.ValidationError({'password2': 'Пароли не
совпадают'})
        try:
            validate_password(data['password1'],
User(username=data['username']))
        except DjangoValidationError as error:
            raise serializers.ValidationError({'password1':
list(error.messages)})
        return data

    def create(self, validated_data):
        return User.objects.create_user(validated_data['username'],
password=validated_data['password1'])
```

ПРИЛОЖЕНИЕ 4

Листинг общего сценария клиентской части (файл frontend/js/app.js)

```
var API = 'http://127.0.0.1:8000/api/';

function getToken() {
    return localStorage.getItem('token') || '';
}

function saveLogin(data) {
    localStorage.setItem('token', data.token);
    localStorage.setItem('username', data.username);
}

function clearLogin() {
    localStorage.removeItem('token');
    localStorage.removeItem('username');
}

function isLoggedIn() {
    return getToken() != '';
}

function api(url, method, data) {
    var options = {method: method || 'GET', headers: {}};
    if (getToken()) {
        options.headers['Authorization'] = 'Token ' + getToken();
    }
    if (data !== undefined) {
        options.headers['Content-Type'] = 'application/json';
        options.body = JSON.stringify(data);
    }
    return fetch(API + url, options).then(function (response) {
        if (response.status == 401) {
            clearLogin();
        }
        if (response.status == 204) {
            return {status: 204, data: {}};
        }
        return response.json().then(function (json) {
            return {status: response.status, data: json};
        });
    });
}

function needLogin() {
    if (!isLoggedIn()) {
        location.href = 'login.html?next=' +
encodeURIComponent(location.pathname.split('/').pop() + location.search);
        return true;
    }
    return false;
}

function getParam(name) {
    return new URLSearchParams(location.search).get(name) || '';
}
```

```
function safe(text) {
    var div = document.createElement('div');
    div.textContent = text;
    return div.innerHTML;
}

function formatPrice(value) {
    value = Number(value);
    var text = value % 1 === 0 ? String(value) :
value.toFixed(2).replace('.', ',');
    return text + ' ₽';
}

function formatDate(text) {
    var date = new Date(text);
    return date.toLocaleString('ru-RU', {day: 'numeric', month: 'long', year:
'numeric', hour: '2-digit', minute: '2-digit'});
}

function pop(element) {
    element.classList.remove('pop');
    void element.offsetWidth;
    element.classList.add('pop');
}


var toasts = document.querySelector('.toasts');

function hideToast(toast) {
    if (toast.classList.contains('is-leaving')) {
        return;
    }
    toast.classList.add('is-leaving');
    setTimeout(function () {
        toast.remove();
    }, 300);
}

function showToast(text, type) {
    var toast = document.createElement('div');
    toast.className = 'toast toast--' + (type || 'success');
    toast.textContent = text;
    toasts.appendChild(toast);

    var all = toasts.querySelectorAll('.toast:not(.is-leaving)');
    if (all.length > 3) {
        hideToast(all[0]);
    }
    setTimeout(function () {
        hideToast(toast);
    }, 2500);
}

function showError() {
    showToast('Сервер не отвечает, проверьте, что бэкенд запущен', 'error');
}

toasts.addEventListener('click', function (event) {
    if (event.target.classList.contains('toast')) {
        hideToast(event.target);
    }
});
```

```
var saved = sessionStorage.getItem('toast');
if (saved) {
    sessionStorage.removeItem('toast');
    showToast(saved);
}

function toastNextPage(text) {
    sessionStorage.setItem('toast', text);
}


function updateCount(count) {
    var counter = document.querySelector('[data-cart-count]');
    if (counter) {
        counter.textContent = count > 0 ? count : '';
        pop(counter);
    }
}

function drawNav() {
    var nav = document.querySelector('.nav');
    var html = '<a href="index.html" class="nav__link">Меню</a>';
    if (isLoggedIn()) {
        html += '<a href="orders.html" class="nav__link">Мои заказы</a>';
        html += '<a href="cart.html" class="nav__cart">Корзина <span
class="nav__count" data-cart-count></span></a>';
        html += '<button type="button" class="link-button" id="logout">' +
safe(localStorage.getItem('username')) + ' · Выйти</button>';
    } else {
        html += '<a href="login.html" class="nav__link">Войти</a>';
        html += '<a href="register.html" class="btn btn--
small">Регистрация</a>';
    }
    nav.innerHTML = html;

    if (isLoggedIn()) {
        document.getElementById('logout').addEventListener('click', function
() {
            api('auth/logout/', 'POST').catch(function ()
{}).finally(function () {
                clearLogin();
                toastNextPage('Вы вышли из аккаунта');
                location.href = 'index.html';
            });
        });
        api('auth/me/').then(function (answer) {
            if (answer.status == 200) {
                updateCount(answer.data.cart_count);
            } else {
                clearLogin();
                drawNav();
            }
        }).catch(function () {});
    }
}

drawNav();


function showReveal() {
    var items = document.querySelectorAll('.hero, .page-title, .menu-
section__title, .card, .panel, .order-card, .empty');
    if (!('IntersectionObserver' in window)) {
        items.forEach(function (item) {
```

```
            item.classList.add('is-visible');
        });
        return;
    }
    var observer = new IntersectionObserver(function (entries) {
        entries.forEach(function (entry) {
            if (!entry.isIntersecting) {
                return;
            }
            var item = entry.target;
            item.classList.add('is-visible');
            observer.unobserve(item);
            setTimeout(function () {
                item.style.transitionDelay = '';
            }, 900);
        });
    }, {threshold: 0.1});

    items.forEach(function (item) {
        if (item.classList.contains('is-visible')) {
            return;
        }
        var index = Array.prototype.indexOf.call(item.parentNode.children,
item);
        item.style.transitionDelay = (index % 4) * 80 + 'ms';
        observer.observe(item);
    });
}


function passwordEyes() {
    document.querySelectorAll('input[type=password]').forEach(function
(input) {
        var wrap = document.createElement('div');
        wrap.className = 'password';
        input.parentNode.insertBefore(wrap, input);
        wrap.appendChild(input);

        var button = document.createElement('button');
        button.type = 'button';
        button.className = 'password__toggle';
        button.title = 'Показать пароль';
        button.innerHTML = '<span class="eye"></span>';
        wrap.appendChild(button);

        button.addEventListener('click', function (event) {
            event.preventDefault();
            if (input.type == 'password') {
                input.type = 'text';
                button.classList.add('is-open');
                button.title = 'Скрыть пароль';
            } else {
                input.type = 'password';
                button.classList.remove('is-open');
                button.title = 'Показать пароль';
            }
            input.focus();
        });
    });
}

function showMessages(box, errors) {
    var html = '';
    errors.forEach(function (error) {
```

```
        html += '<div class="message message--error">' + safe(error) +
'</div>';
    });
    box.innerHTML = html;
}
```